

DILLORA BY KASHVIN

Developer Architecture Guide

Frontend · Backend · Admin · Database · Cloudinary · Razorpay

Technical reference for maintaining & extending the platform

1. System overview

Dillora is a monorepo of three independent apps that talk over one REST API. The backend is the single source of truth; both front-ends are stateless readers/writers.

```
Frontend (Next.js :3000) --public reads--+
Dashboard (Next.js :3001) --JWT writes----+
                                     V
                             Backend REST API (Express :5000)
                             MongoDB · Cloudinary · Razorpay · SMTP
```

2. The three apps

Backend/	Node.js + Express (ESM), Mongoose. Owns data, auth, payments, media, email, invoices.
Frontend/	Next.js (JS/JSX) + Tailwind 3. Customer store. Reads via Frontend/data/api.js. Backend-only (no offline fallback).
dashboard/	Next.js (JS/JSX) + Tailwind 3. Admin panel under app/(dash)/*. JWT-guarded via services/api.js.

3. Backend layout

- **models/** — Mongoose schemas (Product, Order, Coupon, Offer, Brand, Policy, Setting, Invoice, Feedback, Admin...).
- **controllers/** — request handlers (one per resource).
- **routes/** — Express routers mounted in server.js under /api/*.
- **services/** — cross-cutting logic (idService, invoiceService, offerService, couponService, stockService, emailService).
- **config/** — dbConnection.js, cloudinary.js, razorpay.js.
- **middleware/** — authMiddleware (protect / requireRole), error handling.
- **seed/** — seed.js, seedBrands.js, policiesData.js, migrateImages.js.

4. Data model (key collections)

Product	id, name, price, category, stock, gallery, sizes, colorOptions[], relatedIds[], stockLock.
Order	id (DK1), items[], customer, payment, advancePaid/pendingAmount/paymentStatus, cancellation, timeline[].
Setting	single 'site' doc: whatsappNumber, adBanners[], popups{welcome,order,promo}, hero, saleCountdown, delivery, invoice, reviewReward, feedbackPopup.
Coupon / Offer	Coupon = code-based; Offer = auto-applied (BOGO/percent/flat).

Brand / Policy

Brand = phone brands+models+logo; Policy = per-category store policies.

5. API contract & auth

- Public GET endpoints power the store (products, categories, brands, settings, policy, reviews, feedback).
- Writes (POST/PUT/DELETE) require an admin JWT: `Authorization: Bearer <token>`.
- `Token = jwt.sign({id: adminId}, JWT_SECRET)` (utils/generateToken.js); verified in `authMiddleware.protect`.
- Roles: owner / manager / staff via `requireRole(...)`; stored on `Admin.role`.
- Order creation is public (POST /api/orders) — used by COD; online orders are created only after Razorpay signature verification.

6. Database — one DB rule

Critical: the app must use exactly one database, **dillora**. Atlas URIs that omit the DB path make Mongoose default to **test**, splitting data. `config/dbConnection.js` pins it:

```
mongoose.connect(uri, { dbName: process.env.MONGO_DB_NAME || 'dillora' })
Boot logs Connected to MongoDB database: "dillora" and warns on mismatch. All seed scripts share this helper, so they can never write to test.
```

7. Cloudinary (media)

- `config/cloudinary.js` — `uploadBuffer()` compresses (q_auto, width cap) + generates eager thumbnail/card sizes; `optimized()` injects f_auto,q_auto into delivery URLs.
- Admin uploads (products, banners, popups, review media) go to Cloudinary; the original seed product photos are local `/images/*` files served from the storefront.
- Dashboard must set `NEXT_PUBLIC_STORE_URL` so local `/images/*` thumbnails resolve (else 404 on the admin domain).

8. Razorpay (payments)

- `config/razorpay.js` reads keys lazily (ESM import order vs dotenv).
- Flow: Frontend -> POST /api/payment/order (creates Razorpay order for the pay-now amount) -> Razorpay Checkout -> POST /api/payment/verify (HMAC-SHA256 signature check) -> order persisted.
- `paymentBreakdown(payment, total)`: 'online' = fully paid; 'half-cod' = 50% advance; 'cod' = Rs.0 now, full pending (covers-only, enforced server-side).
- Test keys `rzp_test_*`; switch to live keys + enable methods in the Razorpay dashboard for production.

9. Where key features live

COD (covers-only)	orderController.createOrder guard + paymentBreakdown; checkout/page.jsx availablePayments filter.
Colour variants	Product.colorOptions[]; PDP dropdown in product/[id]/page.jsx; admin Variants page.
Cross-sell	Product.relatedIds[]; GET /api/products/:id/related; admin Pair It With.
Popups/Banners	Setting.popups / Setting.adBanners; components ContentPopup, HowToOrderPopup, AdBanners.
Invoices	services/invoiceService.js (pdfkit, single-page); auto-issued on order create/verify.

10. Making common changes fast

Add a field to a product:

- Add it to models/Product.js (and toJSON strip if it defaults to undefined).
- createProduct/updateProduct already spread the body — usually no controller change.
- Surface it in the dashboard product form and on the storefront PDP.

Add an admin-managed setting (e.g. a new toggle):

- Add it to Setting schema (with a default) + handle it in settingController.
- Restart the backend (schema is loaded at boot).
- Read it via useSettings() on the store; edit it on the dashboard Settings page.

Add a new admin page:

- Create dashboard/src/app/(dash)/<name>/page.jsx and add it to the NAV array in (dash)/layout.jsx.
- Add API methods to services/api.js (they attach the JWT automatically).

11. Conventions & gotchas

- Restart the backend after any Mongoose schema change.
- Tailwind arbitrary bg-[linear-gradient(...)] can render transparent — use inline style={{background:...}} for popup/banner card backgrounds.
- Define React components at module scope, not inside other components (react-hooks/static-components).
- Store WhatsApp numbers with the country code (91...) — waLink() strips non-digits but does not add it.
- Dashboard: after big Tailwind/CSS edits, clear dashboard/.next if classes don't apply.

12. Deployment

- Store + dashboard on Vercel; backend on a Node host; MongoDB Atlas + Cloudinary managed.
- Set env per app (see README). Backend MONGO_URI must include /dillora (or MONGO_DB_NAME=dillora).
- Dashboard needs NEXT_PUBLIC_STORE_URL; both front-ends need NEXT_PUBLIC_API_URL.
- Update FRONTEND_URL (CORS) when adding a custom domain. Rotate any secret ever committed to git.